DOCUMENTO DE PLANIFICACIÓN TÉCNICA

Plan de Implementación Arquitectónica

Tienda Virtual Multicanal: arquitectura modular, SOLID y limpia, preparada para crecer y desplegarse en Oracle Cloud

Proyecto base analizado: Tienda-virtual-main (v5.3.0)
Kotlin 1.9 · Ktor 2.3 · Exposed · PostgreSQL · Flyway
Fecha de elaboración: 6 de octubre de 2026 · Versión 1.0

Contenido

1. Resumen ejecutivo . . . 3

2. Diagnóstico de partida . . . 4

2.1 Hallazgos priorizados . . . 4

2.2 Fortalezas que se deben conservar . . . 5

3. Arquitectura objetivo . . . 5

3.1 Principios rectores . . . 5

3.2 Regla de dependencias . . . 5

3.3 Mapa de módulos: de la situación actual al objetivo . . . 6

3.4 Estructura de carpetas objetivo . . . 7

3.5 Catálogo de puertos . . . 7

3.6 Casos de uso de la capa de aplicación . . . 8

3.7 Composición y registro de dependencias . . . 8

3.8 Construcción y versiones . . . 8

4. Persistencia y portabilidad de datos . . . 9

4.1 La base de datos como única fuente de verdad . . . 9

4.2 Aislar lo específico de cada motor . . . 9

4.3 Opciones de base de datos en Oracle . . . 10

4.4 Migraciones por motor . . . 10

4.5 Pruebas de persistencia realistas . . . 10

4.6 Migración de datos desde Supabase . . . 10

5. Despliegue en Oracle Cloud Always Free . . . 11

5.1 Recursos y supuestos . . . 11

5.2 Topología . . . 12

5.3 Dimensionamiento de memoria (referencia sobre 12 GB) . . . 12

5.4 Contenedores . . . 12

5.5 Red y seguridad . . . 13

5.6 Integración y despliegue continuos . . . 13

5.7 Respaldos y recuperación . . . 14

5.8 Observabilidad . . . 14

6. Plan de implementación por fases . . . 14

F0 · Red de seguridad y base de build (2 días) . . . 15
F1 · Correcciones críticas (4 días) . . . 15
F2 · Capa de aplicación y retiro del estado global (7 días) . . . 16
F3 · Modularización de Gradle y de la capa web (4 días) . . . 16
F4 · Portabilidad de datos y pruebas realistas (5 días) . . . 17
F5 · Contenedores y despliegue en Oracle (4 días) . . . 17
F6 · Operación, observabilidad y endurecimiento (3 días) . . . 17
F7 · Evolución y evaluación de Oracle Database (opcional) . . . 18

7. Guía de extensibilidad . . . 18

7.1 Camino de escalabilidad . . . 18

8. Estrategia de calidad . . . 19

8.1 Definición de terminado por fase . . . 19
8.2 Métricas de salud técnica . . . 19

9. Riesgos y mitigaciones . . . 20

10. Próximos pasos inmediatos . . . 20

Elemento	Detalle
Alcance	Plan para evolucionar el código existente hacia una arquitectura modular limpia (hexagonal + SOLID), con persistencia como única fuente de verdad y despliegue portable a Oracle Cloud Always Free.
Base analizada	8 módulos Gradle, unas 9.600 líneas de Kotlin (7.036 en producción y 2.594 en pruebas) y 11 migraciones SQL.
Metodología	Cambios mínimos y verificables por fase: cada fase conserva el sistema funcionando y se puede revertir de forma independiente.
Convención de entrega	Por cada fase: archivos completos modificados, manual de cambios (partes exactas y motivo) y diagrama de clases actualizado en archivo separado.

1. Resumen ejecutivo

El proyecto ya tiene una base sólida: dominio separado en módulos, dinero modelado como entero, un motor de negociación que protege el precio mínimo, controles de seguridad con política de fallo cerrado, patrón outbox e idempotencia de eventos. Lo que lo limita hoy no es el diseño de fondo, sino tres costuras que impedirían crecer con seguridad: **(a)** parte del estado crítico vive solo en memoria, **(b)** la capa web concentra lógica de negocio y **(c)** el despliegue no está pensado todavía para un servidor propio.

Este plan propone resolverlas por fases, sin reescribir lo que funciona. Se toman cinco decisiones de arquitectura que gobiernan todo lo demás:

1. **La base de datos es la única fuente de verdad.** Desaparecen el estado en memoria (`AppStore`, mapa de reservas del pipeline) y los candados locales; la concurrencia se resuelve en la base de datos. Esto es también lo que permitirá, más adelante, ejecutar más de una instancia.
2. **Se introduce una capa de aplicación (casos de uso)** entre las rutas HTTP y el dominio. Las rutas solo traducen HTTP a casos de uso; el negocio se prueba sin levantar un servidor.
3. **La regla de dependencias se verifica con pruebas automáticas** de arquitectura, de modo que ningún cambio futuro pueda romper la separación por descuido.
4. **Portabilidad por contenedores.** La aplicación se empaqueta como imagen para arm64 y amd64, y se despliega con Docker Compose en la máquina virtual de Oracle junto con PostgreSQL. Oracle Autonomous Database queda como alternativa evaluada, con el SQL específico de cada motor aislado en un solo lugar.
5. **Entrega incremental.** Siete fases cortas, cada una con criterios de aceptación medibles y reversible con una etiqueta de Git.

> **Resultado esperado**
> Un sistema donde agregar un canal, un marketplace, un proveedor de IA o una pasarela de pago implica crear un módulo adaptador y registrarlo, sin tocar el dominio ni las rutas; que sobrevive a reinicios sin perder reservas; y que se despliega en Oracle Cloud con un solo comando. Esfuerzo estimado de las fases F0 a F6: unos 29 días de trabajo (6 a 7 semanas a tiempo parcial).

2. Diagnóstico de partida

Los hallazgos se obtuvieron leyendo el código del dominio, el pipeline de mensajes, la seguridad del tablero, el módulo principal y la infraestructura de datos. No se ejecutaron las pruebas en el entorno de análisis, por lo que la prioridad de cada punto se basa en lectura de código y debe confirmarse con una prueba antes de corregir.

2.1 Hallazgos priorizados

ID	Hallazgo	Ubicación	Riesgo	Fase
H1	Las reservas viven en un mapa en memoria, pero el stock reservado sí se persiste. Tras un reinicio el stock queda retenido para siempre y no se puede confirmar el pago. La tabla `tienda.reservas` y las funciones SQL de reserva ya existen (V008, V010) pero el pipeline no las usa.	InboundMessagePipeline	**Crítico**	F1
H2	Limpieza de candados con condición de carrera: se eliminan mutex libres que otra corrutina puede estar a punto de adquirir, y dos corrutinas acaban con mutex distintos para la misma clave.	sweepExpired()	**Alto**	F1
H3	Estado global mutable (`AppStore`) con sincronización dispersa y una segunda fuente de verdad frente a PostgreSQL.	Application.kt	**Alto**	F2
H4	El estado de la reserva difiere entre Kotlin (`CONFIRMED_SALE`) y SQL (`CONFIRMED`, `CONSUMED`).	Reservation.kt / V010	**Medio**	F1
H5	Saga anti-sobreventa con `expectedVersion = 0L` fijo (el control optimista no actúa) y dos ramas idénticas en el mismo `if`.	AntiOversellSaga	**Medio**	F1
H6	Código inalcanzable dentro del bloque `pm >= pp` (el caso ya se resolvió antes).	NegotiationEngine	**Bajo**	F1
H7	Archivos con demasiadas responsabilidades: rutas, negocio y cableado en 1.034 líneas; HTML del tablero como texto en 882 líneas.	Application.kt, DashboardRenderer.kt	**Medio**	F2, F3
H8	36 usos de `System.err`/`println` y ningún logger, aunque Logback ya es dependencia. Un fallo al crear el repositorio se traga en silencio y degrada a memoria.	Varios	**Medio**	F0
H9	Dockerfile con JDK 21 mientras el build exige toolchain 17 sin resolvedor de toolchains: el build en contenedor probablemente falle (a confirmar).	Dockerfile, build.gradle.kts	**Medio**	F0
H10	Versiones de librerías repetidas por módulo; dependencia de H2 en el classpath de producción sin referencias en el código de infraestructura.	build.gradle.kts	**Bajo**	F0, F3
H11	Política CSP con `unsafe-inline` y estilos desde CDN; adecuado para pruebas, débil para producción.	Application.kt	**Bajo**	F6
H12	Lógica de negocio en funciones y triggers de PostgreSQL (reservar, confirmar pago, historial, ventas): potente, pero dificulta cambiar de motor.	V006 a V011	**Medio**	F4

2.2 Fortalezas que se deben conservar

- Estructura multimódulo con puertos y adaptadores ya definida; el dominio no depende de Ktor ni de Exposed.
- Dinero como Long en COP (`Money`, `SignedCop`) y pruebas de fuga del precio mínimo.
- Seguridad con política de fallo cerrado, verificación HMAC, defensa CSRF y límite de intentos de acceso.
- Idempotencia de eventos, patrón outbox y migraciones versionadas con Flyway.
- Cobertura de pruebas de unas 2.600 líneas frente a 7.000 de producción y documentación interna abundante.

3. Arquitectura objetivo

3.1 Principios rectores

Principio	Aplicación concreta en este proyecto
S · Responsabilidad única	Una ruta HTTP por funcionalidad; un caso de uso por intención de negocio; el pipeline deja de decidir y de persistir a la vez.
O · Abierto/cerrado	Nuevos canales, marketplaces o proveedores de IA se añaden implementando un puerto existente y registrándolo; no se modifica código estable.
L · Sustitución de Liskov	Cada puerto tiene una suite de contrato que se ejecuta contra todas sus implementaciones (memoria y base de datos).
I · Segregación de interfaces	Puertos pequeños y específicos (por ejemplo, `ReservationRepository` separado de `InventarioRepository`) en lugar de un repositorio único grande.
D · Inversión de dependencias	El dominio define los puertos; la infraestructura los implementa. El cableado ocurre en un único punto (composition root).
Doce factores	Configuración por variables de entorno, logs a stdout, procesos sin estado, servicios de apoyo (base de datos) como recursos intercambiables.

3.2 Regla de dependencias

Las dependencias solo pueden apuntar hacia el dominio. El dominio no conoce a la aplicación; la aplicación no conoce a la web ni a la infraestructura; la infraestructura y la web dependen de las interfaces del dominio. Esta regla se hará cumplir con pruebas automáticas (sección 8).

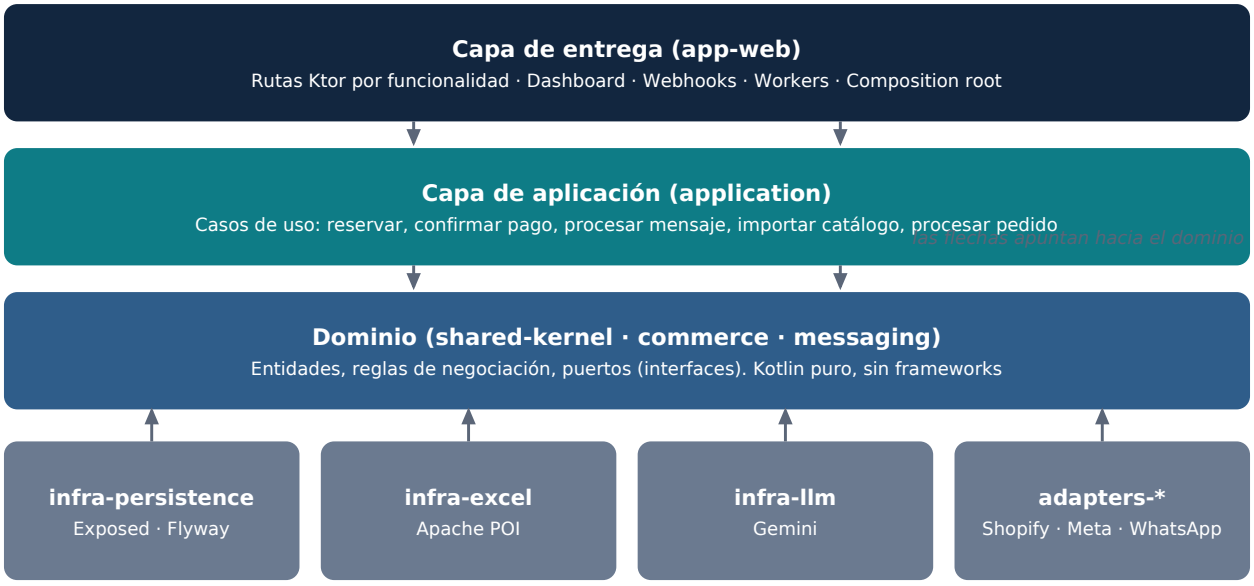


Figura 1. Capas y sentido de las dependencias.

3.3 Mapa de módulos: de la situación actual al objetivo

Módulo actual	Módulo objetivo	Cambio
shared-kernel	shared-kernel	Se mantiene (Money, SignedCop, ProductKey, PortResult, eventos).
commerce	commerce	Se añaden los puertos `ReservationRepository` y `Clock`; se corrigen H4 a H6.
messaging	messaging	Se mantiene; se reubican aquí los contratos de conversación.
(dentro de app)	**application** (nuevo)	Casos de uso extraídos de `InboundMessagePipeline` y de las rutas. Kotlin puro.
infra	**infra-persistence**	Exposed, Flyway, Hikari; implementaciones de repositorios; SQL por motor.
infra	**infra-excel**	ExcelV2Parser, ExcelGenerator, AmountParser (Apache POI).
infra	**infra-llm**	GeminiLlmAdapter detrás de `LlmPort`; permite sumar otros proveedores.
infra	**infra-config**	AppConfig y lectura de entorno; sin dependencias de base de datos.
adapters-shopify, adapters-meta, adapters-channels	Sin cambios de ubicación	Se les añade un registro de adaptadores para descubrirlos sin editar el cableado.
app	**app-web**	Rutas por funcionalidad, vistas del tablero, workers y composition root.

3.4 Estructura de carpetas objetivo

```
tienda-virtual/
  build-logic/                 # plugins de convención Gradle (Kotlin, pruebas, estilo)
  gradle/libs.versions.toml    # catálogo único de versiones
  shared-kernel/
  commerce/                    # modelo, reglas, puertos del negocio
  messaging/                   # conversaciones y puertos de mensajería
  application/                 # casos de uso (sin Ktor, sin Exposed)
    reservas/ ReservarProducto, ConfirmarPago, LiberarVencidas
    mensajes/ ProcesarMensajeEntrante
    catalogo/ ImportarCatalogo, ExportarCatalogo
    pedidos/  ProcesarPedidoShopify
  infra-persistence/           # repositorios Exposed + Flyway
    src/main/resources/db/migration/postgresql/   # V001..V011 (+ nuevas)
    src/main/resources/db/migration/oracle/       # solo si se adopta Oracle DB
  infra-excel/  infra-llm/  infra-config/
  adapters-shopify/  adapters-meta/  adapters-channels/
  app-web/                     # Ktor: rutas/, vistas/, workers/, AppModule.kt
  architecture-tests/          # reglas de dependencias entre módulos
  deploy/                      # Dockerfile, compose, Caddyfile, scripts, CI
```

3.5 Catálogo de puertos

Cada puerto es una interfaz mínima en el dominio. La columna de implementaciones muestra lo que existe hoy y lo que habilita el crecimiento.

Puerto	Responsabilidad	Hoy	Futuro
ReservationRepository (nuevo)	Reservar, confirmar, liberar y expirar con atomicidad	Mapa en memoria	Exposed/PostgreSQL; en memoria para pruebas
InventarioRepository	Inventario y ventas	Exposed	Oracle (dialecto aislado)
ConversationStore	Estado de conversaciones	Memoria y Exposed	Sin cambios
ProcessedEventStore	Idempotencia de eventos	Memoria y Exposed	Sin cambios
MessagingPort / ChannelRouter	Enviar mensajes por canal	WhatsApp, Instagram, Messenger	Telegram u otros canales
CatalogPublishingPort	Publicar catálogo y stock	Shopify, Meta	Otros marketplaces
LlmPort	Interpretar intención del cliente	Gemini (y modo regex)	Otros proveedores
Clock (nuevo)	Obtener la hora actual	Llamadas directas a Clock.System	Reloj inyectable para pruebas deterministas
PaymentPort (nuevo, opcional)	Verificar pagos automáticamente	Confirmación manual	Pasarela de pagos

3.6 Casos de uso de la capa de aplicación

Caso de uso	Reemplaza	Colaboradores (puertos)
ProcesarMensajeEntrante	InboundMessagePipeline.handle	LlmPort, ConversationStore, ReservarProducto, ChannelRouter
ReservarProducto	tryReserve + releaseIfOtherProduct	ReservationRepository, InventarioRepository, Clock
ConfirmarPago	Ruta /api/confirmar-pago y confirmPayment	ReservationRepository, InventarioRepository, VentaRepository
LiberarVencidas	sweep/sweepExpired	ReservationRepository, ConversationStore
ProcesarPedidoShopify	Lógica en la ruta /webhooks/shopify	ProcessedEventStore, InventarioRepository, AntiOversellSaga
ImportarCatalogo / ExportarCatalogo	Lógica en /api/excel/*	ExcelV2Parser, InventarioRepository

3.7 Composición y registro de dependencias

Se crea un único `AppModule` que construye el grafo de objetos (repositorios, adaptadores, casos de uso) y lo entrega a las rutas. Se empieza con inyección manual, sin framework: son pocas dependencias y se evita magia. Si el grafo crece, se puede reemplazar por Koin sin tocar el dominio, porque ningún módulo fuera de `app-web` conoce al contenedor.

Los adaptadores de canal y de catálogo se registran en listas (`ChannelRouter` y la lista de puertos de publicación ya funcionan así). La regla es: agregar un adaptador implica añadir una línea en `AppModule` y nada más.

3.8 Construcción y versiones

- Catálogo de versiones `libs.versions.toml`: una sola declaración de Ktor, Exposed, Flyway, corrutinas y pruebas.
- Plugins de convención en `build-logic`: toolchain, JUnit, estilo y análisis estático se declaran una vez.
- Toolchain alineado con el Dockerfile (se propone JDK 21 LTS en ambos) y resolvedor de toolchains declarado.
- Retirar H2 del classpath de producción; si se necesita para pruebas, pasa a `testImplementation` (o se sustituye por Testcontainers, sección 4.5).

4. Persistencia y portabilidad de datos

4.1 La base de datos como única fuente de verdad

El hallazgo H1 es la prioridad: la reserva debe ser un registro durable. El esquema ya incluye la tabla `tienda.reservas` (identificador, conversación, producto, cantidad, estado, vencimiento, referencia de pedido) y funciones atómicas, de modo que el trabajo consiste en conectar el pipeline a lo que ya existe mediante un puerto, no en diseñar desde cero.

Contrato propuesto para `ReservationRepository`:

Operación	Garantía	Idea de implementación portable
tryReserve(conversación, producto, cantidad, vigencia)	Atómica: o reserva y registra, o no hace nada	Una transacción: UPDATE condicional del inventario (solo si hay disponible) y luego INSERT de la reserva; si el UPDATE no afecta filas, no hay stock.
confirm(reserva)	Idempotente	Transición de estado ACTIVE a CONFIRMED con condición en el WHERE; repetirla no cambia nada.
release(reserva)	Idempotente	Transición a CANCELLED y devolución de stock en la misma transacción.
expireDue(ahora)	Procesa por lotes y sin duplicar	Selección de vencidas con bloqueo de fila saltando las ya tomadas, en lotes pequeños.
findActiveByConversation(id)	Lectura consistente	Consulta indexada por conversación y estado.

Con esto se eliminan los candados en memoria: la exclusión mutua pasa a ser la de la fila de inventario dentro de la transacción. Es la misma garantía, pero válida para más de una instancia y tras reinicios. Además, se unifican los estados de reserva entre Kotlin y SQL (hallazgo H4) con una migración y una enumeración única.

4.2 Aislar lo específico de cada motor

Característica actual	Riesgo al cambiar de motor	Estrategia
Funciones PL/pgSQL (reservar, confirmar_pago, vender_unidades, liberar_vencidas)	Reescritura completa en PL/SQL si se adopta Oracle	La lógica nueva vive en la capa de aplicación con SQL estándar; las funciones existentes quedan como detalle de la implementación PostgreSQL y se retiran gradualmente.
Triggers de historial y de ventas	Distinto dialecto de triggers	Se conservan solo para auditoría; el registro de ventas pasa a un caso de uso explícito.
Row Level Security (V001)	Concepto equivalente distinto (VPD)	La aplicación usa un único rol; el control de acceso se resuelve en la aplicación. RLS se mantiene como defensa adicional en PostgreSQL.
Tipos y sintaxis (TIMESTAMPTZ, TEXT, ON CONFLICT, esquema tienda)	Diferencias de tipos y de upsert	Acceso mediante Exposed (que abstrae dialecto) y SQL manual solo detrás de la interfaz del repositorio.
Conexión sslmode=require	Distinta convención	La validación de conexión segura se vuelve configurable por motor.

4.3 Opciones de base de datos en Oracle

Criterio	A. PostgreSQL en contenedor en la VM	B. Oracle Autonomous Database	C. Mantener Supabase temporalmente
Fidelidad con el código actual	Total: no cambia el SQL ni las funciones	Baja: portar 11 migraciones y funciones a PL/SQL	Total
Esfuerzo	Bajo	Alto (estimado 2 a 3 semanas más)	Ninguno
Costo	Cubierto por la VM Always Free	Cubierto por Always Free (2 instancias de 20 GB)	Según el plan de Supabase
Operación	Tú gestionas parches y respaldos	Gestionada por Oracle	Gestionada por Supabase
Latencia con la app	Mínima (misma máquina)	Baja (misma región)	Mayor (red pública)
Límites a vigilar	Memoria compartida con la app	Cupo de sesiones concurrentes por instancia (verificar vigente)	Pausas por inactividad en planes gratuitos

> **Recomendación**
>
> Adoptar la **opción A** como destino inicial: conserva todo el comportamiento actual con el menor riesgo y mantiene el costo en cero. Usar la opción C solo como etapa transitoria mientras se prepara la VM. Dejar la opción B como una evaluación acotada en la fase F7, sin comprometer el calendario, y solo si se valora más la operación gestionada que la fidelidad.

4.4 Migraciones por motor

Flyway admite ubicaciones por proveedor con el marcador `{vendor}`. Se moverán las migraciones V001 a V011 a `db/migration/postgresql` y se configurará la ubicación como `classpath:db/migration/{vendor}`. Para adoptar otro motor bastará agregar su carpeta; el código de aplicación no cambia. Las migraciones ya aplicadas en producción no se editan; se corrige hacia adelante con nuevas versiones.

4.5 Pruebas de persistencia realistas

- Sustituir la dependencia de H2 por **Testcontainers con PostgreSQL**: H2 oculta diferencias de dialecto justo donde están los riesgos.
- Escribir **suites de contrato** por puerto (por ejemplo, para `ReservationRepository`) y ejecutarlas contra la implementación en memoria y la de base de datos.
- Añadir una **prueba de concurrencia**: N clientes reservando la última unidad al mismo tiempo; debe haber exactamente un ganador.
- Añadir una prueba de **reinicio**: reservar, recrear el pipeline y verificar que la reserva sigue activa y confirmable.

4.6 Migración de datos desde Supabase

1. Congelar escrituras (modo mantenimiento) y registrar el conteo de filas por tabla.
2. Exportar con `pg_dump` en formato personalizado, solo el esquema `tienda` y los datos necesarios.
3. Restaurar en el PostgreSQL de la VM con `pg_restore` y dejar que Flyway valide el historial de migraciones.

4. Ejecutar la función `tienda.verificar_integridad()`, que ya existe en V008, y comparar conteos.
5. Cambiar `DATABASE_URL`, reiniciar y mantener Supabase intacto durante una semana como plan de retorno.

5. Despliegue en Oracle Cloud Always Free

5.1 Recursos y supuestos

La documentación oficial de Oracle indica que Always Free incluye instancias Ampere A1 (Arm) hasta el equivalente de 4 OCPU y 24 GB de memoria, 200 GB de Block Volume y hasta dos Autonomous Database de 20 GB. Algunos artículos de terceros publicados en 2026 mencionan una reducción a 2 OCPU y 12 GB para cuentas nuevas; no pude confirmarlo en la documentación oficial. Por prudencia, **el diseño de este plan cabe en 2 OCPU y 12 GB**, y conviene verificar la cuota real en la consola de OCI antes de empezar.

Aspecto	Consideración	Mitigación
Capacidad de Ampere A1	Es frecuente el error de falta de capacidad al crear la instancia	Reintentar en otro dominio de disponibilidad u otra hora; como contingencia, probar primero en una VM AMD pequeña con la base de datos externa.
Reclamación por inactividad	Oracle puede recuperar instancias Always Free con uso muy bajo sostenido	Revisar los criterios vigentes en la documentación; mantener monitoreo y considerar la cuenta de pago por uso, que conserva los recursos gratuitos, si se necesita garantía.
Arquitectura Arm (arm64)	Las imágenes deben ser compatibles con Arm	Imágenes base oficiales con soporte multiarquitectura; construir con buildx para arm64 y amd64.
Disponibilidad	Una sola VM es un punto único de fallo	Aceptable para esta escala; mitigar con respaldos externos y un runbook de recuperación (sección 5.7).

5.2 Topología

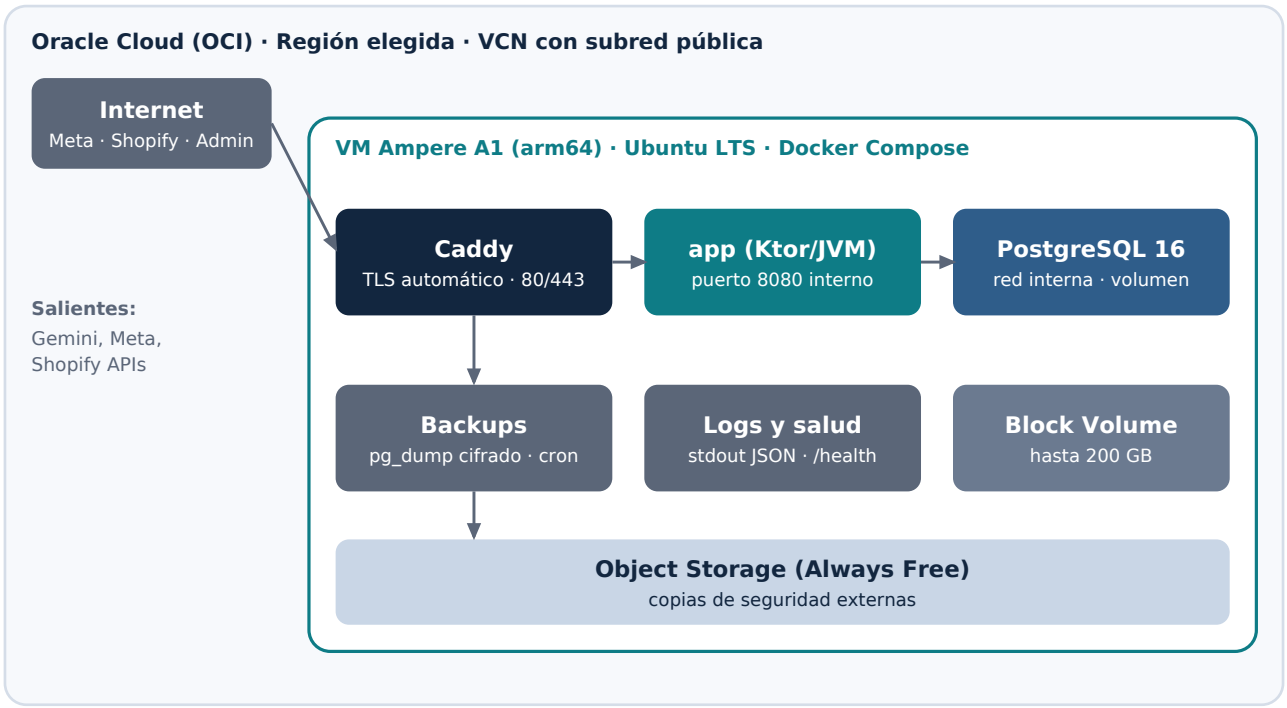


Figura 2. Topología propuesta en una sola VM con contenedores.

5.3 Dimensionamiento de memoria (referencia sobre 12 GB)

Componente	Reserva aproximada	Ajuste principal
Sistema operativo y Docker	1,0 GB	Imagen mínima de Ubuntu LTS
PostgreSQL	1,5 a 2,0 GB	shared_buffers de unos 512 MB; max_connections reducido
Aplicación (JVM)	1,0 a 1,5 GB	Límite de memoria del contenedor y porcentaje máximo de RAM para el heap
Caddy	Menos de 0,1 GB	Sin ajustes
Total	**Menos de 5 GB**	Holgura para picos, caché del sistema y crecimiento

5.4 Contenedores

El Dockerfile actual ya es multietapa y ejecuta como usuario sin privilegios. Se corrige la inconsistencia de JDK (H9), se añade verificación de salud y se construye para arm64. Ejemplo de composición propuesta (los secretos reales van en un archivo `.env` con permisos 600, fuera del repositorio):

```
services:
  caddy:
    image: caddy:2
    ports: ["80:80", "443:443"]
    volumes: ["./Caddyfile:/etc/caddy/Caddyfile:ro", "caddy_data:/data"]
    depends_on: [app]
    restart: unless-stopped
  app:
    image: ghcr.io/TU_USUARIO/tienda:${TAG}
    env_file: .env                  # APP_ENV=production, TRUST_PROXY_HEADERS=true ...
    environment:
      JAVA_TOOL_OPTIONS: "-XX:MaxRAMPercentage=60"
    mem_limit: 1536m
    read_only: true
    tmpfs: ["/tmp"]
    depends_on: { db: { condition: service_healthy } }
    restart: unless-stopped
  db:
    image: postgres:16
    environment: { POSTGRES_DB: tienda, POSTGRES_USER: tienda }
    env_file: .env.db               # POSTGRES_PASSWORD
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck: { test: ["CMD-SHELL", "pg_isready -U tienda"], interval: 10s }
    restart: unless-stopped
volumes: { pgdata: {}, caddy_data: {} }
```

El contenedor de la base de datos no publica puertos hacia el exterior: solo es accesible desde la red interna de Compose. Caddy obtiene y renueva los certificados TLS automáticamente, lo que cumple el requisito de usar el tablero detrás de HTTPS.

5.5 Red y seguridad

- Reglas de la VCN: abrir solo los puertos 80 y 443 al público; SSH restringido a tu IP o mediante el servicio de bastión de OCI.
- Autenticación SSH por llave, sin contraseña; actualizaciones de seguridad automáticas en el sistema operativo.
- Configurar `TRUST_PROXY_HEADERS=true` solo porque la app queda detrás de Caddy, para que el límite de intentos y la defensa CSRF vean la IP y el host reales.
- Los webhooks de Meta y Shopify siguen protegidos por firma; la URL pública con HTTPS se registra en cada plataforma.
- Retirar `unsafe-inline` y el CDN de estilos de la política CSP en la fase F6 (compilar los estilos localmente).
- Rotar los secretos al migrar y almacenarlos fuera del repositorio; evaluar el servicio de secretos de OCI como mejora posterior.

5.6 Integración y despliegue continuos

1. En cada *push*: compilar, ejecutar pruebas (incluidas las de arquitectura y de contrato) y análisis estático.
2. En cada etiqueta de versión: construir la imagen multiarquitectura con buildx y publicarla en un registro (GHCR u OCIR).
3. Despliegue por SSH: `docker compose pull && docker compose up -d`, con verificación de `/health` y reversión automática a la etiqueta anterior si falla.

5.7 Respaldos y recuperación

Elemento	Política
Respaldo lógico	pg_dump diario, comprimido y cifrado, enviado a Object Storage; retención de 7 diarios y 4 semanales.
Objetivo de punto de recuperación	24 horas con la política anterior; mejorable con archivado de WAL si el negocio lo exige.
Objetivo de tiempo de recuperación	Menos de 1 hora con un runbook probado: crear VM, restaurar volumen o dump, levantar Compose.
Verificación	Restauración de prueba trimestral y comprobación con verificar_integridad().

5.8 Observabilidad

- Registros estructurados (JSON) a la salida estándar con SLF4J/Logback, con identificador de correlación por mensaje entrante (H8).
- Endpoints `/health` (la app vive) y `/ready` (base de datos conectada, migraciones al día).
- Un monitor externo gratuito que consulte `/health` y avise por correo o mensaje; métricas con Micrometer como opción.
- Alertas mínimas: reservas vencidas sin liberar, cola de outbox creciente y errores de envío por canal.

6. Plan de implementación por fases

Las fases están ordenadas para reducir riesgo primero (corregir lo crítico sobre una red de seguridad de pruebas) y modularizar después. Los esfuerzos son estimaciones de un desarrollador; el calendario asume dedicación parcial.

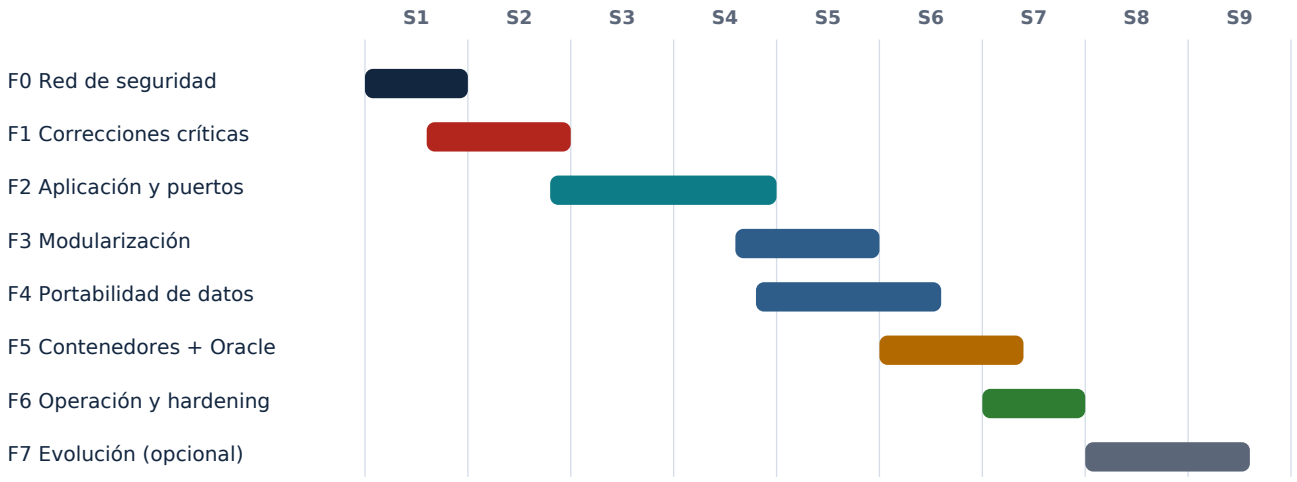


Figura 3. Calendario orientativo en semanas.

Fase	Objetivo	Esfuerzo	Depende de
F0	Red de seguridad y base de build	2 días	n/a
F1	Correcciones críticas (H1, H2, H4, H5, H6)	4 días	F0
F2	Capa de aplicación y retiro del estado global	7 días	F1

Fase	Objetivo	Esfuerzo	Depende de
F3	Modularización de Gradle y de la capa web	4 días	F2
F4	Portabilidad de datos y pruebas realistas	5 días	F1
F5	Contenedores y despliegue en Oracle	4 días	F0, F4
F6	Operación, observabilidad y endurecimiento	3 días	F5
F7	Evolución y evaluación de Oracle Database (opcional)	Continuo	F6

F0 · Red de seguridad y base de build (2 días)

Objetivo. Poder cambiar código con confianza y reproducir el build en cualquier máquina.

Tareas

- Ejecutar la suite actual y registrar la línea base (pruebas verdes, tiempos).
- Crear el catálogo de versiones y alinear el toolchain con el Dockerfile; confirmar o descartar H9.
- Sustituir `println`/`System.err` por un logger SLF4J; registrar el fallo al crear el repositorio de base de datos en lugar de ignorarlo.
- Mover H2 a dependencias de prueba o eliminarla.
- Configurar integración continua mínima (compilar y probar).

Entregables. Archivos de build, Dockerfile y clases con cambios de logging; manual de cambios y diagrama de clases.

Criterios de aceptación

- El build en Docker termina correctamente.
- No quedan llamadas a println en código de producción.
- La integración continua se ejecuta en verde.

F1 · Correcciones críticas (4 días)

Objetivo. Eliminar el riesgo de perder reservas y de doble venta.

Tareas

- Escribir primero las pruebas que reproducen H1 (reinicio) y H2 (concurrencia); deben fallar antes de corregir.
- Crear `ReservationRepository` en el dominio y su implementación Exposed sobre `tienda.reservas`, con reserva atómica.
- Unificar estados de reserva entre Kotlin y SQL con una migración nueva (H4).
- Retirar el mapa de reservas y los candados por conversación y producto del pipeline; recuperar al arrancar las reservas activas desde la base.
- Corregir la saga (versión esperada real y rama duplicada, H5) y el código inalcanzable del motor (H6).
- Hacer idempotente la confirmación de pago, apoyada en la transición de estado.

Entregables. Pipeline, saga, motor de negociación, repositorio de reservas, migración V012 y pruebas nuevas.

Criterios de aceptación

- Tras reiniciar, las reservas siguen activas y confirmables.
- Prueba de concurrencia: con una unidad y 50 clientes hay exactamente una reserva.
- Confirmar el pago dos veces descuenta stock una sola vez.

F2 · Capa de aplicación y retiro del estado global (7 días)

Objetivo. Sacar el negocio de las rutas y del pipeline monolítico, y eliminar H3.

Tareas

- Crear el módulo `application` y mover allí los casos de uso de la sección 3.6, uno por vez, con sus pruebas.
- Reemplazar `AppStore` por repositorios; si se necesita rendimiento de lectura para el tablero, usar una caché de solo lectura con invalidación explícita.
- Introducir el puerto `Clock` para volver deterministas las pruebas de tiempo.
- Dividir `InboundMessagePipeline` en pasos pequeños (interpretar, decidir, reservar, responder) detrás de `ProcesarMensajeEntrante`.

Entregables. Nuevo módulo application, pipeline reducido, rutas delgadas y diagrama de clases con la capa nueva.

Criterios de aceptación

- Las rutas no contienen reglas de negocio.
- No existe estado mutable global.
- Las pruebas del negocio corren sin Ktor ni base de datos.

F3 · Modularización de Gradle y de la capa web (4 días)

Objetivo. Que cada módulo tenga una razón de cambio y dependencias mínimas.

Tareas

- Dividir `infra` en persistencia, Excel, LLM y configuración.
- Separar `Application.kt` en archivos de rutas por funcionalidad y un `AppModule` como composition root.
- Mover las vistas del tablero a su propio paquete y partir `DashboardRenderer` por pantallas.
- Añadir el módulo de pruebas de arquitectura con las reglas de dependencia.

Entregables. Archivos de build y de rutas reorganizados, composition root y pruebas de arquitectura.

Criterios de aceptación

- Ningún archivo de producción supera unas 300 líneas.
- Las pruebas de arquitectura fallan si el dominio importa Ktor o Exposed.

F4 · Portabilidad de datos y pruebas realistas (5 días)

Objetivo. Dejar la persistencia lista para cambiar de servidor o de motor sin sorpresas.

Tareas

- Mover las migraciones a `db/migration/postgresql` y configurar la ubicación con `{vendor}`.
- Incorporar Testcontainers con PostgreSQL y suites de contrato por puerto.
- Trasladar al código de aplicación la lógica nueva de reservas y ventas; marcar las funciones SQL antiguas como obsoletas.
- Preparar y ensayar el procedimiento de migración desde Supabase (sección 4.6).

Entregables. Migraciones reubicadas, suites de contrato, guía de migración probada en un entorno de ensayo.

Criterios de aceptación

- Las suites de contrato pasan con ambas implementaciones.
- El ensayo de migración termina con conteos iguales y verificar_integridad() sin errores.

F5 · Contenedores y despliegue en Oracle (4 días)

Objetivo. Tener el sistema funcionando en la VM de Oracle de forma reproducible.

Tareas

- Verificar la cuota de Always Free y crear la VM Ampere A1, la VCN y las reglas de red.
- Instalar Docker, preparar Compose, Caddy y el dominio con DNS.
- Construir la imagen multiarquitectura y publicar en el registro; configurar el despliegue automático.
- Registrar las URL de webhook de Meta y Shopify y probar de extremo a extremo.

Entregables. Carpeta `deploy/`, flujo de CI/CD y guía de puesta en marcha.

Criterios de aceptación

- El tablero responde por HTTPS con certificado válido.
- Un mensaje real de WhatsApp recorre el flujo completo hasta la reserva.
- Reiniciar la VM deja el sistema operativo sin intervención.

F6 · Operación, observabilidad y endurecimiento (3 días)

Objetivo. Operar con tranquilidad: ver lo que pasa, recuperarse de fallos y reducir superficie de ataque.

Tareas

- Implementar `/health` y `/ready`, registros JSON y monitor externo.
- Automatizar respaldos a Object Storage y ensayar una restauración.
- Endurecer la política CSP y revisar cabeceras, puertos y permisos.
- Escribir el runbook de incidentes y recuperación.

Entregables. Scripts de respaldo, runbook, ajustes de seguridad y documentación operativa.

Criterios de aceptación

- Restauración probada en menos de 1 hora.

- Una alerta llega al caer la aplicación.
- La CSP no requiere `unsafe-inline`.

F7 · Evolución y evaluación de Oracle Database (opcional)

Objetivo. Aprovechar la arquitectura nueva para crecer y decidir con datos sobre Oracle Database.

Tareas

- Prototipo acotado en una Autonomous Database: portar el esquema mínimo y ejecutar las suites de contrato.
- Incorporar nuevos canales o marketplaces siguiendo las recetas del capítulo 7.
- Evaluar una cola de mensajes dedicada solo si el volumen lo justifica.

Entregables. Informe comparativo con esfuerzo real, límites encontrados y recomendación.

Criterios de aceptación

- Decisión documentada (adoptar, aplazar o descartar) con datos medidos.

7. Guía de extensibilidad

El valor de esta arquitectura se mide por lo poco que hay que tocar al crecer. Las siguientes recetas muestran el alcance esperado de cada ampliación una vez completadas las fases F2 y F3.

Quiero añadir…	Pasos	Qué NO se toca
Un canal de mensajería (por ejemplo, Telegram)	1) Nuevo módulo `adapters-telegram` que implemente `MessagingPort`. 2) Receptor de webhook en `app-web` que traduzca el evento a `ProcesarMensajeEntrante`. 3) Una línea en `AppModule`. 4) Pruebas del adaptador.	Dominio, negociación, reservas, persistencia.
Un marketplace (por ejemplo, MercadoLibre)	1) Módulo adaptador que implemente `CatalogPublishingPort`. 2) Mapeo del identificador externo en `publicacion_canal`. 3) Registro en la lista de puertos de la saga.	Reglas de negocio; la saga ya itera sobre todos los puertos.
Otro proveedor de IA	1) Clase en `infra-llm` que implemente `LlmPort`. 2) Selección por variable de entorno en `AppModule`.	Pipeline y reglas de precios (la IA solo interpreta texto).
Una pasarela de pagos	1) Puerto `PaymentPort` y adaptador. 2) Webhook que invoque `ConfirmarPago`. 3) Mantener la confirmación manual como alternativa.	Reservas, inventario y tablero.
Una pantalla nueva del tablero	1) Vista en el paquete de vistas. 2) Ruta delgada que llame a un caso de uso de lectura.	Dominio y repositorios existentes.

7.1 Camino de escalabilidad

Con la base de datos como fuente de verdad, la aplicación queda sin estado local, y eso abre el camino a crecer por etapas sin rediseñar:

1. **Una VM.** Suficiente para el volumen esperado de una tienda semiautogestionada.
2. **Más recursos o dos instancias** de la aplicación tras el proxy; las reservas son seguras porque la exclusión la da la base de datos.
3. **Base de datos gestionada o réplica de lectura** si el tablero compite con las escrituras.

4. **Multitienda:** una columna de inquilino en las tablas y un contexto de tienda por solicitud, aprovechando que el acceso a datos ya pasa por repositorios.
5. **Cola de mensajes dedicada** solo si el outbox basado en base de datos deja de ser suficiente.

8. Estrategia de calidad

Nivel	Qué verifica	Herramienta sugerida
Unitarias de dominio	Negociación L0 a L4, dinero, estados; reloj inyectado	JUnit 5 (ya presente)
Casos de uso	Reglas de negocio con puertos en memoria	JUnit 5 con implementaciones en memoria
Contrato de puertos	Todas las implementaciones se comportan igual	Suites abstractas de JUnit 5 + Testcontainers
Concurrencia y recuperación	Última unidad con muchos clientes; reinicio con reservas activas	Corrutinas de prueba + Testcontainers
Arquitectura	El dominio no importa Ktor ni Exposed; la aplicación no importa infraestructura	Konsist o ArchUnit
Estilo y análisis estático	Formato y problemas comunes	ktlint y detekt
Extremo a extremo	Webhook firmado, mensaje, reserva, pago confirmado	Prueba de integración con servidor de prueba de Ktor

8.1 Definición de terminado por fase

- Compila, las pruebas pasan y las pruebas de arquitectura están en verde.
- Los cambios están en una rama con etiqueta de Git que permite revertir la fase completa.
- El manual de cambios lista cada parte modificada y el motivo; el diagrama de clases está actualizado.
- No se introduce ninguna dependencia nueva sin justificarla en el manual.

8.2 Métricas de salud técnica

Métrica	Hoy (aprox.)	Meta
Archivo de producción más grande	1.034 líneas	Menos de 300 líneas
Estado mutable global	Presente (AppStore)	Ninguno
Llamadas a println/System.err	36	0
Reservas que sobreviven a un reinicio	No	Sí
Módulos con dependencia prohibida	No medido	0 verificado automáticamente

9. Riesgos y mitigaciones

Riesgo	Prob.	Impacto	Mitigación
La reforma de reservas introduce una regresión en ventas reales	Media	Alto	Pruebas que reproducen el fallo antes de corregir; despliegue primero en ensayo; reversión por etiqueta.
No hay capacidad de Ampere A1 al crear la VM	Alta	Medio	Reintentos programados; plan alterno con otra región o VM temporal; la app ya es portable por contenedores.
La cuota real de Always Free es menor a la esperada	Media	Medio	Verificar en consola; el diseño cabe en 2 OCPU y 12 GB.
Pérdida de datos por falla de la VM única	Baja	Alto	Respaldos cifrados diarios fuera de la VM y restauración ensayada.
Complejidad de portar la lógica de PostgreSQL a otro motor	Media	Medio	Opción A por defecto; lógica nueva en la aplicación; evaluación de Oracle Database acotada en F7.
Sobreingeniería (demasiados módulos para el tamaño del equipo)	Media	Medio	Dividir solo donde hay razón de cambio distinta; revisar tras F3 y fusionar módulos que no aporten.
Cambios de políticas del proveedor gratuito	Media	Medio	Contenedores y respaldos portables; documentación de salida hacia otro proveedor.
Secretos expuestos durante la migración	Baja	Alto	Rotación al migrar, archivos con permisos restringidos y fuera de Git.

10. Próximos pasos inmediatos

1. Confirmar la cuota de Always Free y la región en la consola de Oracle Cloud.
2. Aprobar el alcance de la fase F0 y de la fase F1, que son las que reducen el riesgo operativo.
3. Indicar si se prefiere mantener Supabase de forma transitoria durante F5 o migrar la base de datos de una vez.
4. Con esa confirmación, se entrega la fase F0: archivos completos modificados, manual de cambios y diagrama de clases actualizado.

> **Nota sobre las cifras de Oracle**
>
> Los límites del nivel gratuito cambian con el tiempo. Antes de ejecutar la fase F5, validar en la documentación oficial de Always Free y en la consola de la cuenta los recursos disponibles, los criterios de reclamación por inactividad y los límites de Object Storage y de Autonomous Database.